

算法与算法复杂性理论 - 作业 4

1. 分支定界法求 TSP 问题

根据题目给出的数据，将错位的字符修正后，初始费用矩阵 D 为：

$$D = \begin{pmatrix} \infty & 5 & 13 & 18 & 9 \\ 20 & \infty & 14 & 3 & 6 \\ 25 & 11 & \infty & 4 & 7 \\ 6 & 3 & 13 & \infty & 8 \\ 19 & 5 & 16 & 22 & \infty \end{pmatrix}$$

第一步：初始矩阵规约

对每一行减去该行的最小值（分别为 5, 3, 4, 3, 5），得到行规约矩阵：

$$\begin{pmatrix} \infty & 0 & 8 & 13 & 4 \\ 17 & \infty & 11 & 0 & 3 \\ 21 & 7 & \infty & 0 & 3 \\ 3 & 0 & 10 & \infty & 5 \\ 14 & 0 & 11 & 17 & \infty \end{pmatrix}$$

对每一列减去该列的最小值（分别为 3, 0, 8, 0, 3），得到完全规约矩阵 M_0 ：

$$M_0 = \begin{pmatrix} \infty & 0 & 0 & 13 & 1 \\ 14 & \infty & 3 & 0 & 0 \\ 18 & 7 & \infty & 0 & 0 \\ 0 & 0 & 2 & \infty & 2 \\ 11 & 0 & 3 & 17 & \infty \end{pmatrix}$$

根节点的初始下界 $LB_0 = (5 + 3 + 4 + 3 + 5) + (3 + 0 + 8 + 0 + 3) = 20 + 14 = 34$ 。

第二步：分支搜索（从城市 1 出发）

从节点 1 分支到 2, 3, 4, 5。

- **1 → 2:** $M_0[1, 2] = 0$ 。划去第 1 行和第 2 列，设 $M[2, 1] = \infty$ 。规约后需进一步在第 5 行减去 3，产生规约成本 3。 $LB_{12} = 34 + 0 + 3 = 37$ 。
- **1 → 3:** $M_0[1, 3] = 0$ 。划去第 1 行和第 3 列，设 $M[3, 1] = \infty$ 。剩余矩阵无需额外规约，规约成本 0。 $LB_{13} = 34 + 0 + 0 = 34$ 。

- **1 → 4:** $M_0[1,4] = 13$ 。划去第 1 行第 4 列, 设 $M[4,1] = \infty$ 。后续规约成本 13。 $LB_{14} = 34 + 13 + 13 = 60$ 。
- **1 → 5:** $M_0[1,5] = 1$ 。划去第 1 行第 5 列, 设 $M[5,1] = \infty$ 。后续规约成本 2。 $LB_{15} = 34 + 1 + 2 = 37$ 。

当前最小下界为节点 **1 → 3** ($LB = 34$), 继续对其分支。

第三步: 从 **1 → 3** 继续分支

当前规约矩阵为 M_{13} 。可选下一步为 2, 4, 5。

- **1 → 3 → 2:** $M_{13}[3,2] = 7$ 。划去第 3 行第 2 列, 设 $M[2,1] = \infty$ 。产生规约成本 11。 $LB_{132} = 34 + 7 + 11 = 52$ 。
- **1 → 3 → 4:** $M_{13}[3,4] = 0$ 。划去第 3 行第 4 列, 设 $M[4,1] = \infty$ 。产生规约成本 11。 $LB_{134} = 34 + 0 + 11 = 45$ 。
- **1 → 3 → 5:** $M_{13}[3,5] = 0$ 。划去第 3 行第 5 列, 设 $M[5,1] = \infty$ 。矩阵无需额外规约。 $LB_{135} = 34 + 0 + 0 = 34$ 。

选择最小下界 **1 → 3 → 5** ($LB = 34$) 继续分支。

第四步: 从 **1 → 3 → 5** 继续分支

可选下一步为 2, 4。

- **1 → 3 → 5 → 2:** 边成本为 0, 规约成本为 0。 $LB_{1352} = 34$ 。此时仅剩 $2 \rightarrow 4 \rightarrow 1$, 形成完整回路。

结论

我们找到了一条完整路径: $1 \rightarrow 3 \rightarrow 5 \rightarrow 2 \rightarrow 4 \rightarrow 1$ 。验证总费用: $D[1,3] + D[3,5] + D[5,2] + D[2,4] + D[4,1] = 13 + 7 + 5 + 3 + 6 = 34$ 。由于 34 恰好等于当前所有活动分支节点中的最小下界 (其他未探索分支的 LB 均 ≥ 37), 因此该路径即为最优解。

总费用最小路径: $1 \rightarrow 3 \rightarrow 5 \rightarrow 2 \rightarrow 4 \rightarrow 1$

总费用: 34

2. MAX-SAT 问题的分支定界算法

设给定的子句集 Φ 共有 m 个子句, 包含 n 个布尔变量 $x_1, x_2, \dots, x_n$ 。

状态空间树定义: 构建一棵深度为 n 的二叉树, 第 i 层代表对变量 x_i 的赋值 (True 或 False)。从根节点 (无变量赋值) 出发, 遍历到叶子节点即为一种完整的真值指派。

分支定界算法设计:

1. **全局变量:** 维护一个全局最优解 `best_count`, 初始化为 0。
2. **节点状态维护:** 在搜索树的任一节点 (设已对 $x_1 \dots x_k$ 赋值), 维护两个值:

- `sat_count`: 在当前部分赋值下, 已经必定为 `True` 的子句数量。
- `falsified_count`: 在当前部分赋值下, 所有文字均已赋值为 `False`, 从而必定为 `False` 的子句数量。

3. **定界 (Bounding) 与剪枝**: 对于当前节点, 其能满足的最大子句数上限为:

$$UB = m - \text{falsified_count}$$

如果 $UB \leq \text{best_count}$, 说明即使后续所有未确定子句都为真, 当前分支也无法超越已知最优解, 因此直接**剪枝**。

4. **搜索策略 (分支)**: 采用深度优先搜索 (DFS)。在第 k 层:

- 分支一: 令 $x_k = \top$, 更新 `sat_count` 和 `falsified_count`, 向下递归。
- 分支二: 令 $x_k = \perp$, 更新 `sat_count` 和 `falsified_count`, 向下递归。

5. **叶子节点处理**: 当递归到第 $n+1$ 层 (所有变量均已赋值) 时, 更新最优解: `best_count = max(best_count, sat_count)`。

3. 证明 First Fit 是近似比为 2 的近似算法

证明. 设物品集合为 $S = \{s_1, s_2, \dots, s_n\}$, 其中 $0 < s_i \leq 1$ 。设最优装箱算法使用的箱子数为 OPT 。因为每个箱子容量为 1, 显然最优箱子数不可能小于所有物品体积之和, 即:

$$OPT \geq \sum_{i=1}^n s_i$$

设 First Fit 算法实际使用的箱子数为 k 。如果 $k = 1$, 由于 $OPT \geq 1$, 显然 $k \leq 2OPT$ 成立。

假设 $k \geq 2$ 。根据 First Fit 算法的规则: 当一个物品被放入新箱子时, 说明它无法放入前面的任何一个已打开的箱子。因此, 对于 First Fit 打开的任意两个不同的箱子 B_i 和 B_j (假设 $i < j$), 箱子 B_i 的剩余容量必定小于箱子 B_j 中放入的第一个物品的体积。这意味着, 箱子 B_i 中物品的总体积与箱子 B_j 中第一个物品的体积之和严格大于 1。从而, 任意两个箱子的物品总体积之和必定严格大于 1:

$$V(B_i) + V(B_j) > 1 \quad (\forall i \neq j)$$

将这 k 个箱子两两配对:

- 如果 k 是偶数, 可分为 $k/2$ 对, 总物品体积 $\sum s_i > \frac{k}{2} \times 1 = \frac{k}{2}$ 。
- 如果 k 是奇数, 可分为 $(k-1)/2$ 对外加一个独立的箱子, 总物品体积 $\sum s_i > \frac{k-1}{2} \times 1 = \frac{k-1}{2}$ 。

在所有情况下，都有 $\sum s_i > \frac{k-1}{2}$ 。

结合最优解下界：

$$\begin{aligned} OPT &\geq \sum s_i > \frac{k-1}{2} \\ 2OPT &> k-1 \implies 2OPT \geq k \end{aligned}$$

因此， $k/OPT \leq 2$ 。First Fit 是近似比为 2 的近似算法。 $\square$

4. 背包问题的贪心算法证明

(a) 证明朴素贪心算法的近似比可以任意地差

证明. 考虑如下的背包实例：背包容量限制为 B ($B \geq 2$ 为很大的正整数)。只有两个物品：

- 物品 1：价值 $v_1 = 2$ ，重量 $w_1 = 1$ 。其性价比为 $2/1 = 2$ 。
- 物品 2：价值 $v_2 = B$ ，重量 $w_2 = B$ 。其性价比为 $B/B = 1$ 。

根据朴素的贪心算法（按性价比从大到小排序），排序为：物品 1 $\rightarrow$ 物品 2。算法遍历时，先选取物品 1，消耗重量 1。接着考虑物品 2，由于剩余容量 $B-1 < B$ ，物品 2 无法装入。因此，朴素贪心算法的总价值为 $V_{greedy} = 2$ 。

而最优解显然是仅选取物品 2，其总价值为 $OPT = B$ 。此时，近似比为：

$$\frac{OPT}{V_{greedy}} = \frac{B}{2}$$

对于任意给定的 $N > 1$ ，只需令背包容量 $B > 2N$ ，即可使得该实例的最优解大于贪心算法价值之和的 N 倍。故该贪心算法的近似比可以任意差。 $\square$

(b) 证明改进后的算法是近似比为 2 的近似算法

证明. 设所有物品按价值/重量比降序排列后依次为 $1, 2, \dots, n$ 。设朴素贪心算法按照此顺序依次挑选，其最终选取的物品总价值为 V_{greedy} 。在朴素贪心遍历的过程中，设第 k 个物品是第一个由于重量超出剩余容量而未能被选取的物品 ($w_k \leq B$ ，如果是单件超重则直接忽略不计入有效物品)。

如果在背包问题中允许物品被拆分（即分数背包问题），那么最优的分数背包解 OPT_{frac} 可以通过完全装入前 $k-1$ 个物品，并将物品 k 的一部分装满剩余空间来获得。显然：

$$OPT_{frac} \leq \left(\sum_{i=1}^{k-1} v_i \right) + v_k \leq V_{greedy} + v_k$$

由于 0-1 背包的最优解 OPT 必定不大于分数背包的最优解 OPT_{frac} ，因此有：

$$OPT \leq OPT_{frac} \leq V_{greedy} + v_k$$

根据改进算法的描述，设 v^* 为所有“重量不超过 B 且未被选取”的物品中价值最大的。由于物品 k 属于未被选取的物品，必然有：

$$v_k \leq v^*$$

从而：

$$OPT \leq V_{greedy} + v^*$$

改进后的算法将最终输出方案定为 $\max(V_{greedy}, v^*)$ 。设该算法输出价值为 ALG ，则：

$$V_{greedy} \leq ALG \quad \text{且} \quad v^* \leq ALG$$

因此：

$$OPT \leq V_{greedy} + v^* \leq ALG + ALG = 2ALG$$

即 $OPT/ALG \leq 2$ 。证明改进后的算法是近似比为 2 的近似算法。

□